

Challenge Lab: End-to-End 3-Tier Deployment

PHASE: CAPSTONE PROJECT | FOCUS: DOCKER, K8S MANIFESTS, & NETWORKING

Lab Prompt:

You are tasked with taking raw application code for a 3-tier Todo app (Frontend, Backend API, and Database), packaging the code into Docker containers, and writing declarative YAML manifests to orchestrate the entire architecture on a local Kubernetes cluster.

Lab Sequence

Phase 1 Workspace & Source Code Preparation

Create a repository named `3-tier-k8s-lab`. Set up the following files to serve as your raw application code.

1. `backend/requirements.txt`

```
Flask==2.0.1
psycpg2-binary==2.9.1
flask-cors==3.0.10
```

2. `backend/app.py` (Snippet for API connection logic)

```
import os
from flask import Flask, jsonify, request
from flask_cors import CORS
import psycpg2

app = Flask(__name__)
CORS(app)

def get_db_connection():
    return psycpg2.connect(
        host=os.environ['DB_HOST'],
        database=os.environ['DB_NAME'],
        user=os.environ['DB_USER'],
        password=os.environ['DB_PASSWORD']
    )

# ... standard Flask routes for GET/POST /todos ...
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

3. `frontend/index.html` (Snippet for UI logic)

```
<!DOCTYPE html>
<html>
<body>
  <h2>My DevOps Todo List</h2>
  <!-- JS to fetch tasks -->
  <script>
    // You will update this URL in Phase 4!
    const API_URL = 'http://localhost:5000/todos';
    /* ... fetch logic ... */
  </script>
</body>
</html>
```

Phase 2 Dockerizing the Application

You need to write `Dockerfile` s to package the frontend and backend. The database will use the official `postgres:13-alpine` image, so no Dockerfile is needed for it.

- **Backend Dockerfile:** Use `python:3.9-slim` , copy requirements, run `pip install -r requirements.txt` , copy code, and expose port `5000` .
- **Frontend Dockerfile:** Use `nginx:alpine` and copy `index.html` to `/usr/share/nginx/html` .

Build your images using the following commands:

```
docker build -t todo-api:1.0 ./backend
docker build -t todo-web:1.0 ./frontend
```

Phase 3 K8s Cluster Setup & Database Manifest

Start your local cluster (Minikube). Load your newly built images into the cluster so K8s can see them:

```
minikube image load todo-api:1.0
minikube image load todo-web:1.0
```

Create `k8s/db.yaml` . Use a Deployment and a `ClusterIP` service on port 5432. Inject environment variables into the container spec like this:

```
env:
  - name: POSTGRES_USER
    value: "admin"
  - name: POSTGRES_PASSWORD
    value: "secretpassword"
  - name: POSTGRES_DB
    value: "tododb"
```

Phase 4 Kubernetes API & Frontend Manifests

Create `k8s/api.yaml` . Use your `todo-api:1.0` image. Add environment variables pointing to the database (e.g., `DB_HOST` should be the name of your DB Service). Expose it with a `NodePort` Service.

Create `k8s/web.yaml` . **Crucial Step:** Find out the NodePort K8s assigned to your API by running `kubectl get svc` . Update the `API_URL` inside your `frontend/index.html` to point to your Minikube IP and API NodePort (e.g., `http://192.168.49.2:31000/todos`). Rebuild the frontend Docker image and reload it into Minikube!

Deploy your frontend image and expose it with a `NodePort` Service.

Phase 5

Deployment and Submission

Apply your infrastructure-as-code to the cluster:

```
kubectl apply -f k8s/  
kubectl get pods  
kubectl get svc
```

Verify the app works in your browser. Then, commit your code and open a Pull Request for review.



Helpful Hints for Research

- **ImagePullPolicy:** Because you built `todo-api:1.0` locally, K8s might try to download it from Docker Hub and fail. Add `imagePullPolicy: Never` to your Deployment containers to force K8s to use the local image.
- **K8s DNS:** Inside the K8s network, Pods communicate using Service names. If your database service is named `db-svc`, your backend API should use `db-svc` as its `DB_HOST` environment variable.
- **Client-Side Routing:** Remember that HTML/JS runs in *your web browser*, outside the K8s cluster! That is why the Frontend Javascript must use the API's external `NodePort` address, not its internal K8s name.